



1. Управление процессами — это уровень над отдельным агентом

До этого мы в основном рассматривали, как работает один агент или одна группа субагентов внутри конкретной задачи. Теперь я хочу подняться на уровень выше и посмотреть на систему, в которой одновременно существует несколько работ.

Представим, что один агент анализирует Pull Request, другой проверяет документацию, третий выполняет миграцию, а ещё несколько задач ожидают запуска. Здесь уже недостаточно правильно сформулировать промпт для каждого агента. Нам необходимо решить, какие задачи запускать сейчас, какие должны ждать, сколько исполнителей разрешено использовать одновременно и как понимать, что работа действительно укладывается в ожидаемые сроки.

Именно этот уровень я называю управлением процессами. Мы управляем уже не содержанием одного ответа, а потоком агентных работ.

2. Оркестрация и выполнение задачи — не одно и то же

Оркестратор сам не обязан выполнять всю полезную работу. Его задача состоит в том, чтобы понимать структуру процесса и распределять выполнение между доступными исполнителями.

Мы уже видели простой вариант такой схемы, когда основной Hermes Agent последовательно создавал Planner, Coder и Reviewer. В теме управления процессами эта идея становится шире. Оркестратор может иметь несколько независимых задач и решать, какие из них допустимо выполнить параллельно, а какие зависят от предыдущих результатов.

Hermes поддерживает передачу сразу нескольких независимых задач через механизм delegation. Каждая дочерняя задача получает собственный контекст и терминальную сессию, а основной агент получает итоговые результаты. Для более сложной структуры Hermes также допускает отдельную роль orchestrator, способную создавать следующий уровень субагентов, если такая глубина явно разрешена в конфигурации.

Главный смысл оркестрации заключается не в создании максимально большого количества агентов, а в управлении зависимостями между работами.

3. Очередь задач

Как только задач становится больше, чем система готова выполнять одновременно, появляется очередь.

Очередь я рассматриваю как набор работ, которые уже известны системе, но ещё не обязательно выполняются. У каждой задачи есть состояние. Она может ожидать запуска, выполняться, завершиться успешно или закончиться неуспешно.

Это принципиально отличается от обычного списка пожеланий. Задача в управляемой очереди должна иметь достаточно информации, чтобы система понимала, что именно необходимо выполнить и где находится её рабочий контекст.

В Hermes разные механизмы имеют собственные способы хранения и запуска работ. Делегирование хорошо подходит для подзадач текущей интерактивной сессии, а длительные или запланированные работы могут выполняться через cron jobs. Cron-задачи запускаются в отдельных свежих сессиях и могут получать собственную рабочую директорию проекта.

Поэтому очередь — это уже не просто часть промпта. Она становится частью инфраструктуры агентской системы.

4. Приоритизация задач

Если в очереди находятся десять задач, это ещё не означает, что их нужно выполнять в порядке появления.

Представим, что одна задача связана с документацией, другая — с упавшим production CI, а третья — с плановым обновлением зависимости. Формально все три задачи существуют, но их бизнес-важность совершенно разная.

Приоритизация отвечает на вопрос, какая работа должна получить ресурс раньше.

Я бы не привязывал приоритет только к словам «высокий» или «низкий». При принятии решения нужно учитывать срочность, влияние на другие процессы и наличие зависимостей. Например, если пять последующих задач заблокированы одной проверкой, выполнение этой проверки может быть важнее более старой задачи в очереди.

Для агента приоритет должен быть задан явно или выводиться из заранее определённых правил. Нельзя рассчитывать, что модель всегда сама правильно определит бизнес-критичность задач.

5. Параллельность и ограничения ресурсов

Параллельное выполнение может значительно ускорить процесс, но только если задачи действительно независимы.

Hermes позволяет выполнять несколько дочерних агентов одновременно, однако количество параллельных исполнителей ограничивается настройками delegation. При превышении установленного лимита система не должна бесконтрольно создавать всё новые экземпляры агентов.

Почему это важно? Каждый субагент использует модель, контекст, инструменты и внешние сервисы. Если запустить слишком большое количество работ одновременно, мы можем столкнуться с ограничениями API, ростом стоимости, нагрузкой на CPU или память и конфликтами при работе с общей кодовой базой.

Поэтому хорошая оркестрация всегда предполагает ограниченную параллельность. Максимальное количество агентов — это не цель производительности, а ресурсный лимит.

6. Квоты как способ ограничить потребление ресурсов

Квота — это заранее установленная граница потребления ресурса.

В агентной системе квоты могут относиться к числу одновременных агентов, глубине делегирования, числу итераций или количеству параллельно запускаемых фоновых работ. В Hermes, например, можно ограничивать количество одновременно работающих дочерних агентов и глубину дерева delegation. Для дочерних агентов также существует предел количества итераций.

Это необходимо не только для экономии денег. Квоты защищают систему от неконтролируемого разрастания процесса.

Представим orchestrator, который создаёт трёх агентов, а каждый из них создаёт ещё по три. Через несколько уровней число исполнителей растёт очень быстро. Даже если каждый отдельный агент выполняет разумную задачу, общая система становится дорогой и трудно контролируемой.

Поэтому увеличение лимитов должно быть осознанным архитектурным решением.

7. Backpressure и перегрузка системы

Теперь рассмотрим ситуацию, когда новые задачи поступают быстрее, чем система успевает их выполнять.

Если просто продолжать запускать всё подряд, мы получим перегрузку. В распределённых системах для этой ситуации используется понятие backpressure — механизм, который не позволяет входящему потоку бесконтрольно превышать возможности исполнителей.

В Hermes похожий принцип можно увидеть на уровне API server. Для одновременно запускаемых agent runs может устанавливаться общий предел. Когда он достигнут, новые запросы не начинают выполняться бесконечно параллельно, а получают ответ о превышении допустимого количества одновременных запусков.

На уровне проектирования это означает, что система должна уметь сказать: сейчас ресурсы заняты, задача должна подождать или быть повторена позже.

Это намного безопаснее, чем создавать сотни конкурентных агентов только потому, что задачи поступили одновременно.

8. SLA для агентных процессов

Теперь перейдём к SLA — Service Level Agreement. В нашем контексте я рассматриваю SLA как формализованное ожидание относительно обслуживания задачи.

Например, мы можем определить, что критическая проверка Pull Request должна начаться в течение определённого времени, а обычная документационная задача может ждать значительно дольше.

Важно понимать, что SLA — это не обещание модели «ответить хорошо». Это характеристика процесса.

Для агентной системы можно отдельно измерять время ожидания в очереди, время фактического выполнения и общее время от постановки задачи до получения результата.

Допустим, агент решил задачу за две минуты, но перед этим задача сорок минут ждала свободного исполнителя. Для пользователя время выполнения составило не две, а сорок две минуты.

Поэтому управление SLA невозможно без наблюдения за состояниями задач.

9. Мониторинг статуса

Если у нас работает несколько агентов, недостаточно знать только итоговый ответ.

Мне необходимо понимать, какие задачи ожидают запуска, какие выполняются сейчас и какие уже завершились. Для длительных процессов особенно важно различать активную работу и зависшее состояние.

Hermes предоставляет разные механизмы наблюдения в зависимости от типа процесса. У *delegation* можно отслеживать дочерние задачи и их завершение. У *cron jobs* существует управление состояниями задач, включая просмотр, паузу, возобновление и ручной запуск. Долгоживущие *terminal*-процессы могут выполняться в фоне и отдельно уведомлять о завершении.

Это означает, что мониторинг не должен строиться только на сообщении агента «я работаю». Состояние должно подтверждаться инфраструктурой, которая реально управляет выполнением.

10. Управляемая агентная система

В итоге я хочу, чтобы мы различали две совершенно разные ситуации.

В первой пользователь пишет агенту одну задачу и ждёт результата. Здесь управление процессами почти незаметно.

Во второй существует постоянный поток работ. Одни задачи поступают от пользователей, другие появляются по расписанию, третьи иницируются состоянием внешних систем. Часть работ можно выполнять параллельно, часть должна ждать результатов предыдущих этапов. На все эти процессы расходуются ограниченные вычислительные и финансовые ресурсы.

Именно во второй ситуации появляются оркестрация, очередь, приоритеты, квоты, ограничения параллельности и SLA.

Hermes предоставляет для этого несколько уровней механизмов: `delegate_task` подходит для распределения текущей интеллектуальной работы между субагентами, `background terminal` — для длительных процессов, `cron` — для устойчивых запланированных запусков, а конфигурационные лимиты позволяют ограничивать количество параллельных операций.

Поэтому управление процессами я рассматриваю как отдельный инженерный слой агентной системы. Мы уже умеем создавать агента, давать ему инструменты и подключать внешние знания. Теперь нам необходимо научиться управлять ситуацией, когда таких задач становится много и они конкурируют за одни и те же ресурсы.

В следующей практической работе мы как раз перейдём к этому уровню: создадим несколько задач для агентов, организуем их очередь, зададим порядок выполнения и будем отслеживать изменение статусов непосредственно в Hermes.